



EUROPEAN
**MICROSOFT
FABRIC+SQL**
Community Conference

BARCELONA
28 SEPT - 01 OCT 2026



FABCONSPK

WIFI password: fabcon-spk26



Building a Production-ready Lakehouse in Microsoft Fabric

NIKOLA ILIC

Data Mozart | Microsoft Data Platform MVP



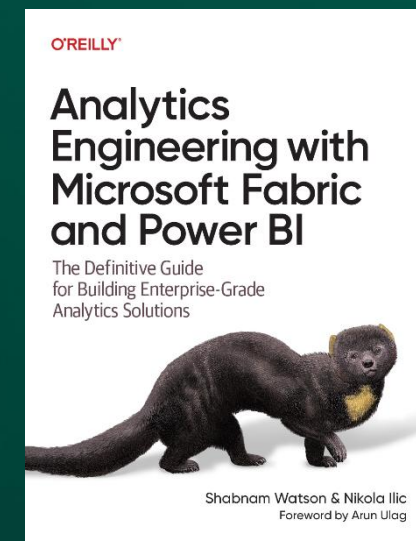
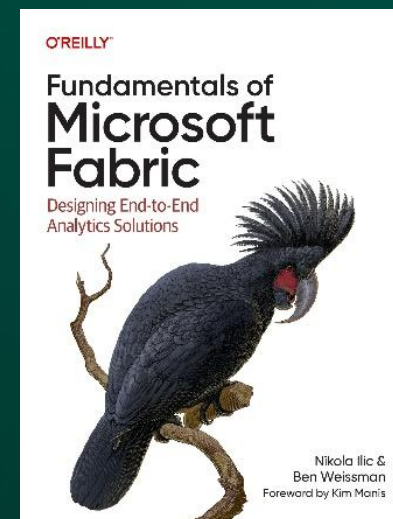
Nikola Ilic

Principal Architect



data-mozart.com

- *I'm making music from the data!*
- Power BI and Fabric addict, blogger, speaker...
- Father of 2, **Barca** & Leo Messi fan...



Proctors



Sue Bayes

Microsoft Data Platform MVP



Miguel Felix

Microsoft Data Platform MVP



Khaled Chowdhury

Principal @ Data Crafters

Special thanks...



Alex Powers

Microsoft | Guy in a Cube

AGENDA

TIME	BLOCK	TYPE
08:30-09:00	Lab 0 - arrival and environment check	Setup
09:00-09:20	Opening: the story and the architecture	Slides
09:20-10:05	Lab 1 - Bronze file ingestion	Hands-on
10:05-10:30	Lab 2 , part 1 - control table and watermark notebook	Hands-on
10:30-11:00	Morning break	
11:00-11:45	Lab 2 , part 2 - child and parent pipelines	Hands-on
11:45-12:15	Lab 3 - API and the day-2 file drop	Hands-on
12:15-12:25	The SharePoint war story	Slides
12:25-12:45	Catch-up and Q&A	Buffer
12:45-14:00	Lunch	
14:00-15:15	Lab 4 - Silver and the late refund	Hands-on
15:15-15:45	Afternoon break	
15:45-16:10	Lab 5 - Gold star schema	Hands-on
16:10-16:25	Lab 6 - optimize for Direct Lake	Hands-on
16:25-16:50	Lab 7 - end-to-end orchestration	Hands-on
16:50-17:00	Debrief: forensics walkthrough , ghost to trusted, production checklist	Slides

MOSAIC EVENTS GROUP

➤Runs conferences across Europe



MOSAIC
EVENTS GROUP

MOSAIC EVENTS GROUP · REPORTED REVENUE, SEASON TO DATE

EUR 2,847,310

Finance does not trust this number.

By the afternoon, you will know they were right.

Mosaic Event Group



Venue point-of-sale

CSV exports, dropped nightly

Events, venues, sessions, products,
check-ins, concessions



Ticketing platform

Azure SQL database (mosaicops)

Attendees, orders, order lines,
payments, refunds



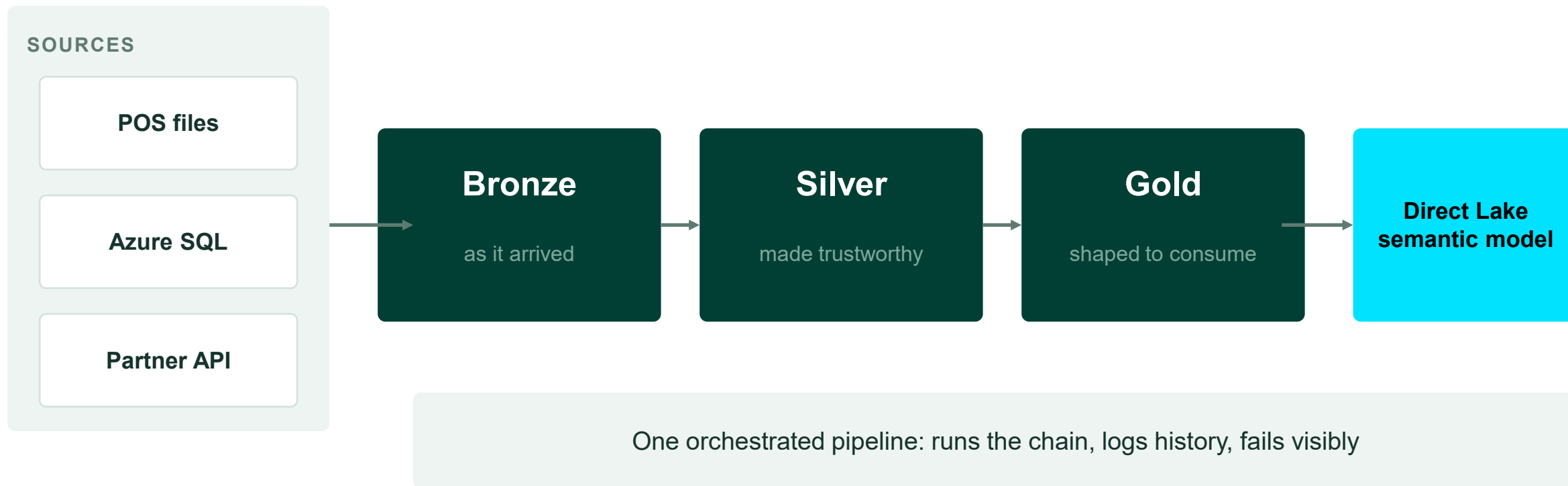
Partner management

Vendor REST API, paginated JSON

Exhibitors, sponsors

Three sources. Three delivery styles. Three different ways to be wrong.

WHAT are we going to build today?



HOW are we going to build it?

The site

Every lab, every code block, copy buttons, your progress. Link on your seat card.

Checkpoints

Each lab ends in an assertion that prints green. Green means safe to continue.

Helpers

Badged and roaming. Stuck for three minutes means raise your hand.

Spares

Broken account or workspace: we swap you in ninety seconds. Nothing is precious.

BEFORE LAB 1 · THE PROBLEM

“Where did this number come from?”

Nobody at Mosaic could answer with a file name and a timestamp. Every investigation started from zero, and most of them ended there.

BRONZE: A tape recorder, not an editor

1 · Append only

Never update, never delete. Bronze records what arrived, mistakes included. Corrections happen downstream.

2 · Provenance stamped

`_load_timestamp`, `_source_file`, `_batch_id` on every row. When, where from, which delivery.

3 · Re-runnable

Ingested files are tracked and skipped. Running the loader twice must not double the data.

A Bronze row remembers

```
sale_id: C-004417  
amount: 12
```

```
_load_timestamp:  
2026-09-01 05:14:02
```

```
_source_file:  
concessions_export_  
2026-08-31.csv
```

```
_batch_id: batch1
```

What provenance buys you?

Auditability

Every number traces to a file and a timestamp

Replayability

Fix the code, rebuild downstream, history intact

Debuggability

Provenance turns days of archaeology into minutes

Source independence

Vendors change; your history doesn't

The cost: storage for duplicate arrivals, and the discipline to never shortcut the layer. Cheap insurance.

HANDS-ON

Lab 1 · Bronze file ingestion

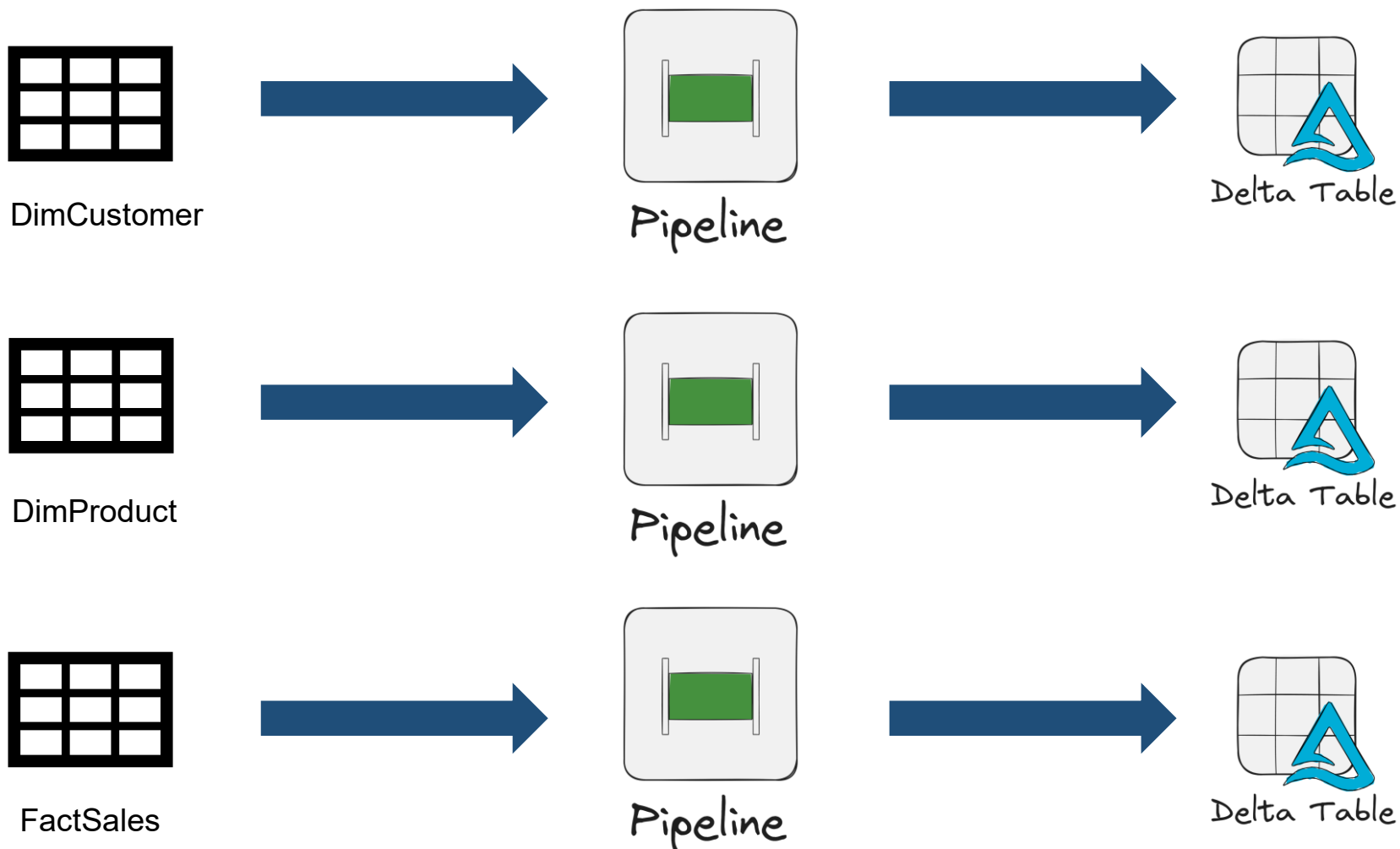
45 minutes · Site → Lab 1 · You are done when the checkpoint prints: Checkpoint 1 green

BEFORE LAB 2 · THE PROBLEM

5 pipelines. Then table 6 arrived...

The prototype had one hardcoded pipeline per SQL table: five pipelines, five places to fix every bug, and a new deployment every time someone added a source.

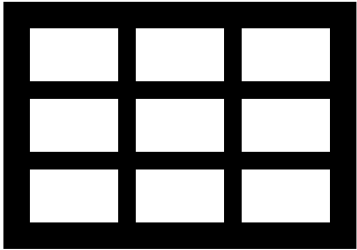
Metadata-driven pipelines



Metadata-driven pipelines

➤ Parameterize everything

➤ Server name



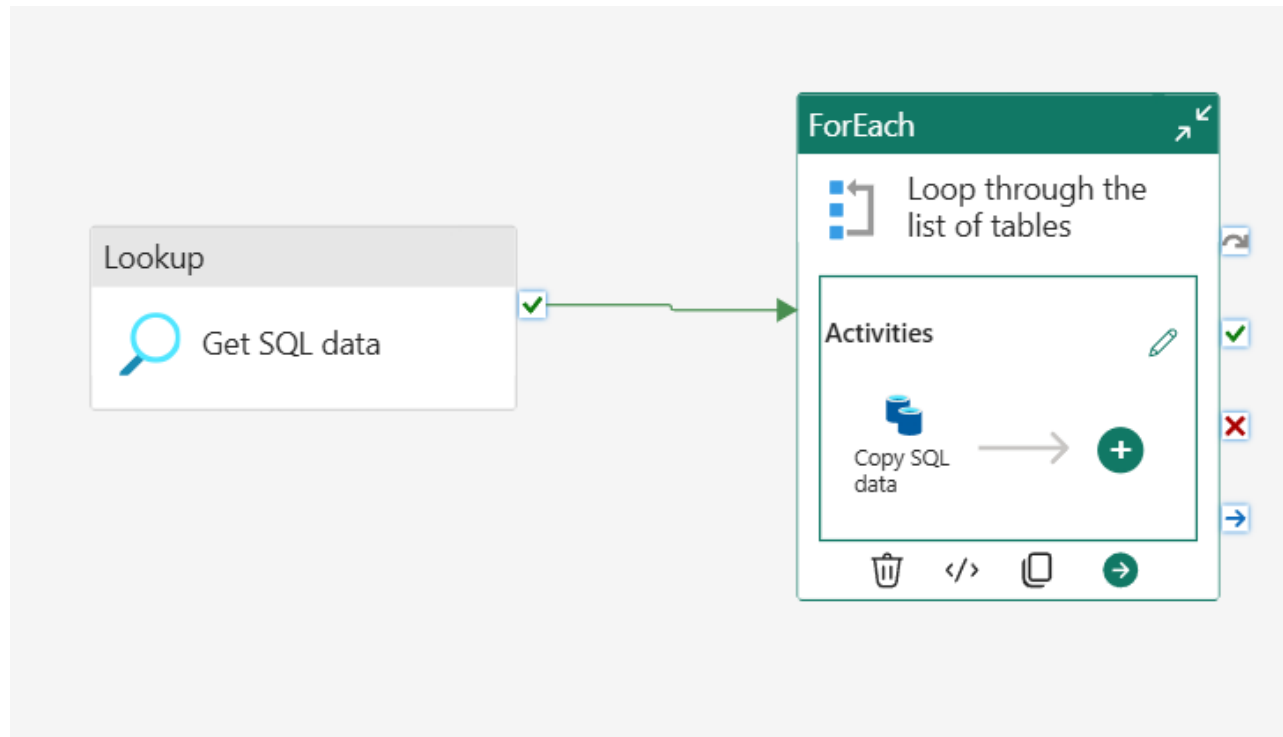
Control table

SourceTable	TargetTable	SourcePath	TransformationScript	LoadType
Customers	DimCustomer	/bronze/customers	clean_customer.sql	Full
Sales	FactSales	/bronze/sales	clean_sales.sql	Incremental

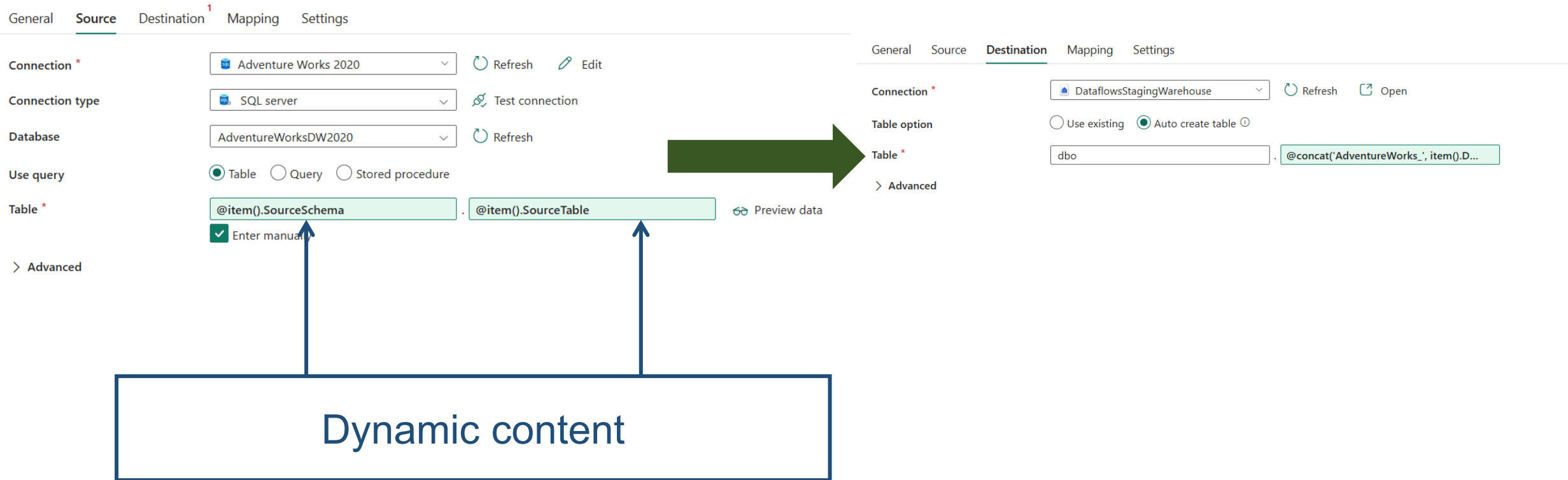
➤ Easily extend with a new line of data -> no need for a new pipeline

➤ Bulk Copy from Database template

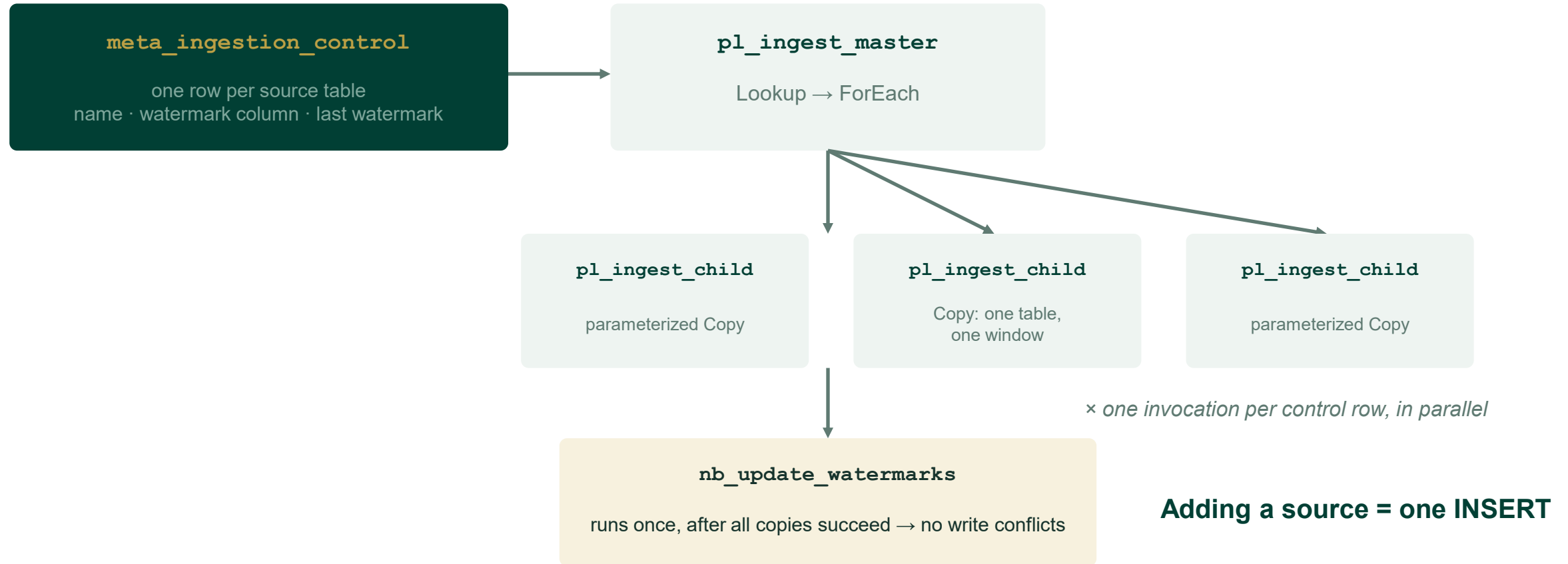
Metadata-driven pipelines



Metadata-driven Pipelines



One engine, one control table



WHY IT EARNS ITS KEEP

Configuration as data

Scale stops hurting

50 tables cost the same engineering as 5

Auditing is a query

What loads, from where, since when: `SELECT *`

Self-service onboarding

Data owners register tables; nobody edits pipelines

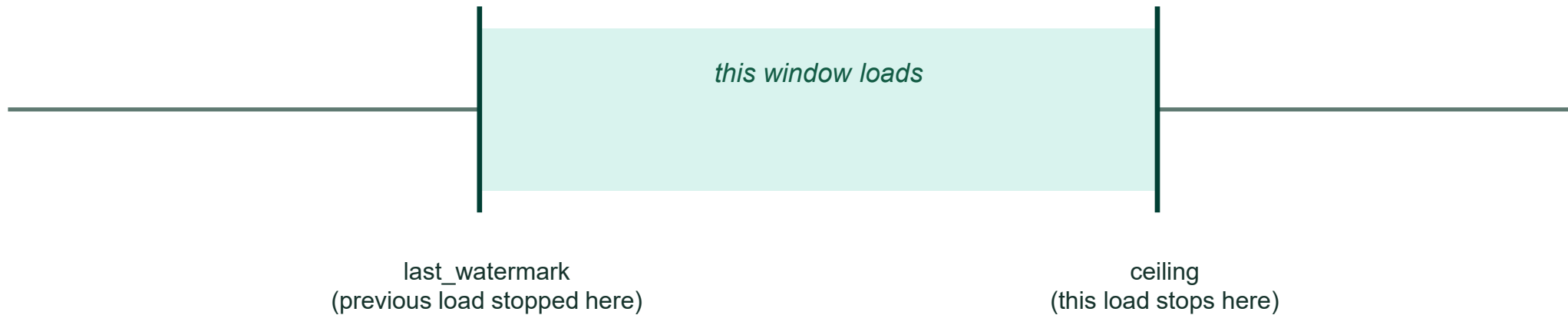
The trade-off

More abstraction up front; config typos replace code bugs

Adding a source table is one INSERT into a control table.

The watermark window

```
WHERE wm > last_watermark AND wm <= ceiling
```



Idempotent

Same window twice → second run copies zero rows

Resumable

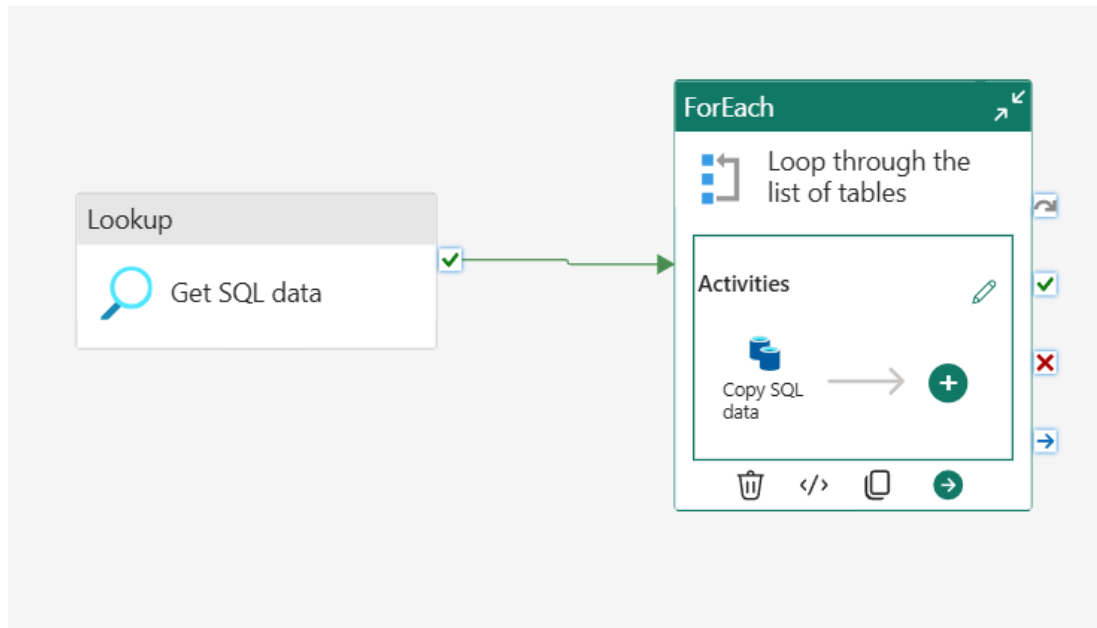
Failed load → watermark stays → next run retries the window

Correct

A row modified after the ceiling waits for the next window

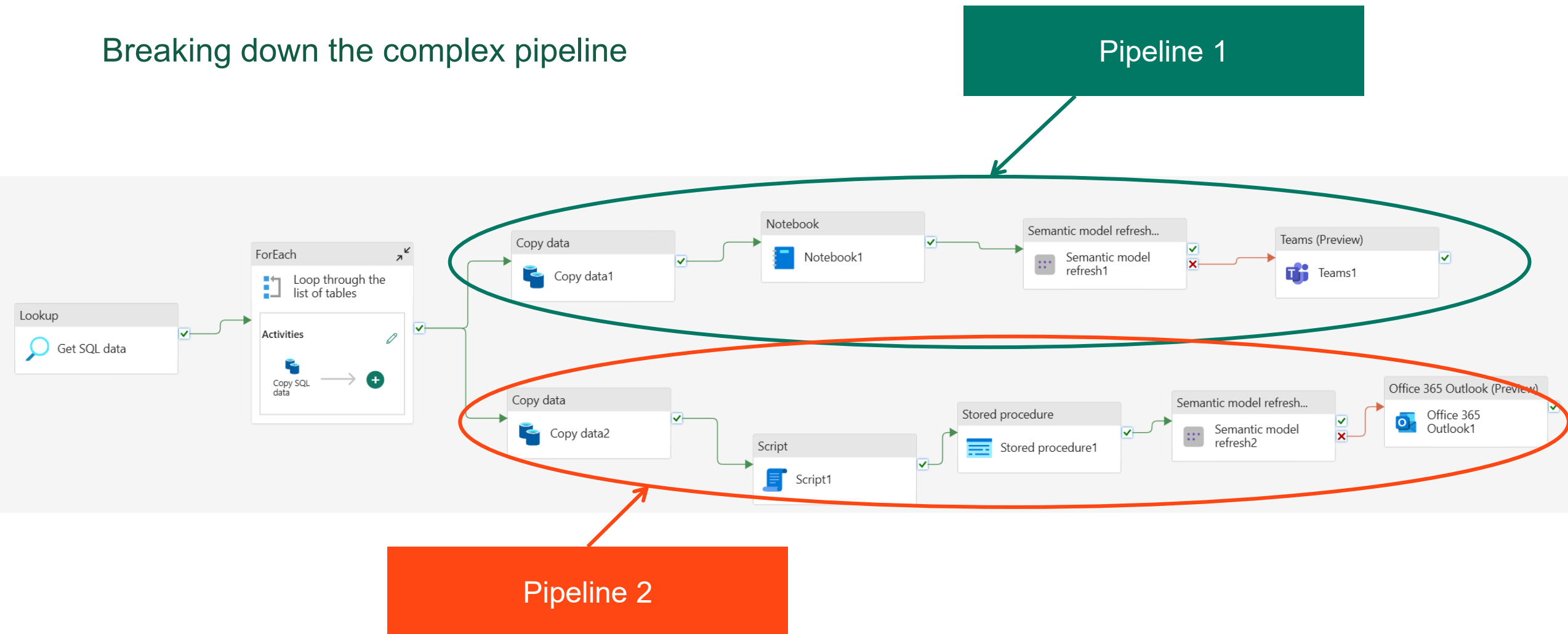
Parent-Child pipelines

Breaking down the complex pipeline



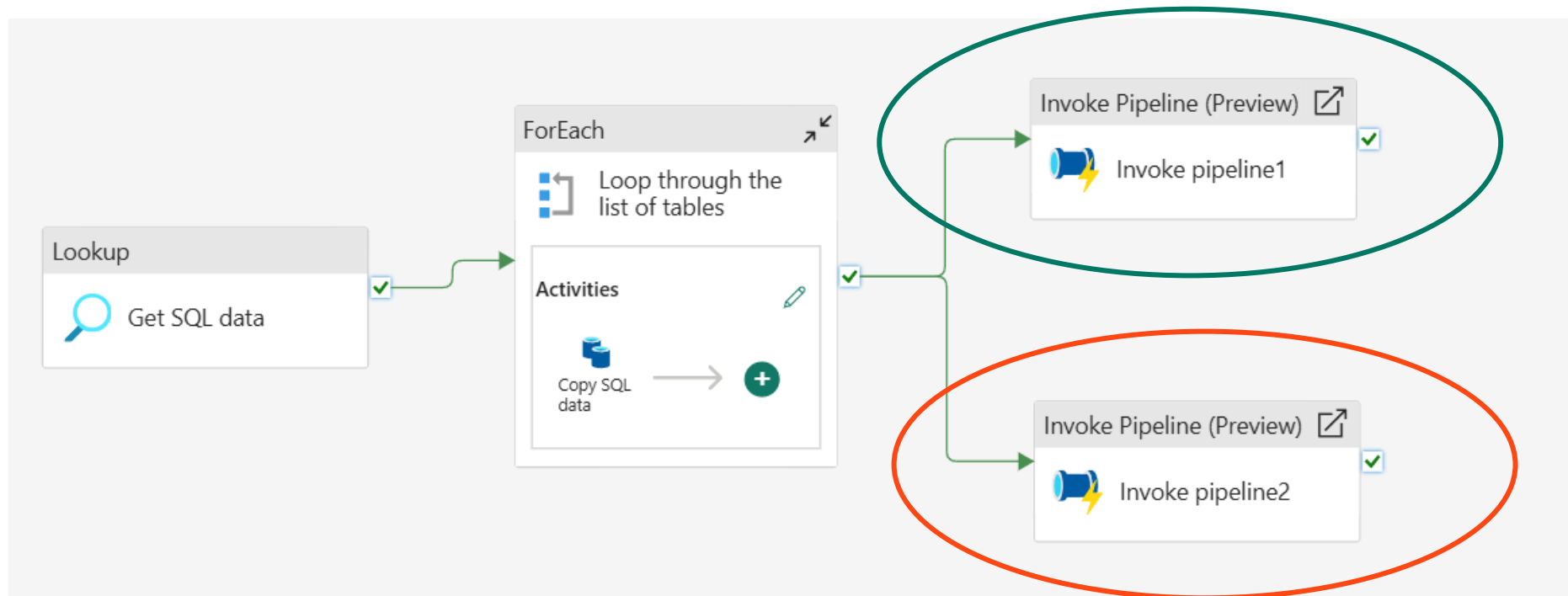
Parent-Child Pipelines

Breaking down the complex pipeline



Parent-Child Pipelines

Breaking down the complex pipeline



➤ Pass parameters from parent to child

HANDS-ON

Lab 2 · The metadata-driven framework

- Part 1 (25 min, before break): control table + watermark notebook
- Part 2 (45 min, after break): child + parent pipelines, demo-along · Checkpoint 2 green

HANDS-ON

Lab 3 · API and the day-2 file drop

- 30 minutes · Checkpoint 3 green

The file that came back with a new name

6,200 day 1 export

5,800 day 2 export

6,200 day 1 again, renamed "... (1).csv"

18,200 rows in Bronze · phantom revenue: EUR 63,625

Your loader honored its contract. The contract was wrong: file identity = file name.

The API that repeated itself politely

81 rows 80 distinct

Valid JSON on every page · correct total_pages · no error, ever

Page 1 ends with EXH-025. Page 2 begins with EXH-025.

The only witness was your assertion: count vs distinct count.

Assert on source contracts. Keep buggy loads in Bronze as evidence.

A TRUE STORY

Three days. One file name.

A manufacturing client, a nightly SharePoint export, a sync job that re-uploaded one file. Every downstream aggregate inflated by a plausible amount for six weeks.

BEFORE LAB 4 · THE PROBLEM

Bronze faithfully recorded every lie.

18,200 concession rows, 6,200 of them phantoms. An API load with a built-in echo. Strings where money should be. Bronze did its job. Now someone has to tell the truth.

The silver contract

Typed

Money is an integer, time is a timestamp. Cast once at the boundary; everyone downstream inherits it.

Deduplicated on business keys

One row per sale_id / order_id / attendee_id, no matter how many times the fact arrived. The renamed file dies here, by contract.

Incremental

Process only Bronze rows newer than the last Silver run. Changed orders get their aggregates recomputed from source.

Mechanism: `MERGE` on the business key. `Matched` → `update`. `Not matched` → `insert`.

Two contracts, two guarantees

File-level idempotency

Lab 1 · guards the pipe

- Unit: the delivery (a file)
- Promise: the same file loads once
- Defeats: re-runs, re-triggers
- Defenseless against: the same data under a new name

Business-key deduplication

Lab 4 · guards the meaning

- Unit: the business fact (a key)
- Promise: the same fact exists once
- Defeats: renamed files, echoing APIs, replays
- The contract that survives SharePoint

HANDS-ON

Lab 4 · Silver and the late refund

- 75 minutes, the longest lab of the day · Checkpoint 4 green
- Typed, deduplicated, incrementally maintained Silver, then the almost-right bug

SCAR 3 · THE ONE YOU JUST HIT

EUR 399

Everyone in this room is off by exactly the same amount.

The extract: correct - one row, the right row, REF-2001

The merge: correct - the refund is sitting in silver.refunds

The aggregate logic: correct - recomputed every changed order

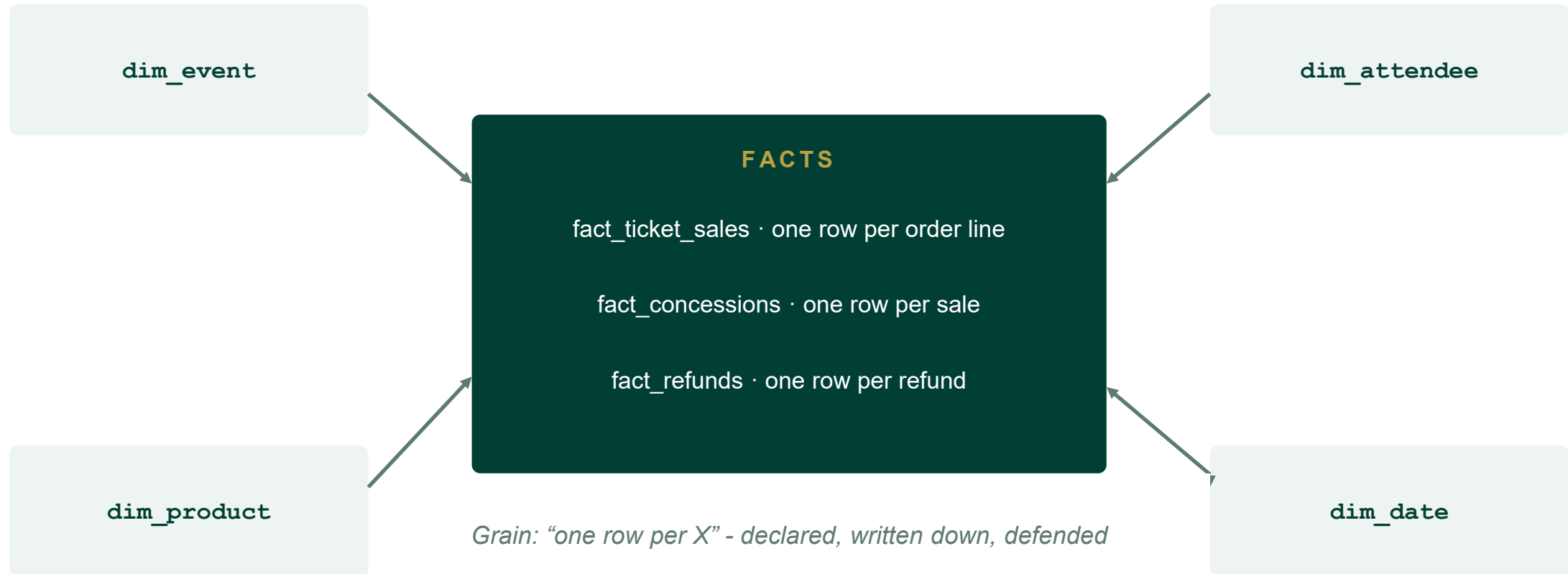
ORD-1042 never changed. Only its refund did. The aggregate was driven by orders and lines - refunds never made the list.

BEFORE LABS 5 AND 6 · THE PROBLEM

The data is finally true. The report is still wrong.

Net revenue was defined in four different reports, four different ways. The Power BI model read operational tables and took ninety seconds to open. Correct data, consumed badly, is still a broken platform.

Facts, dimensions, and one declared grain



net revenue = a MEASURE (gross – refunds), defined once in the model. Never a stored column.

CONCEPT

Direct Lake reads your files. Your layout is your speed.



OPTIMIZE

compacts many small files into few large ones → fewer reads at framing



V-Order

rewrites parquet sorted and dictionary-friendly → leaner columns in memory



VACUUM

removes unreferenced files · retention ≥ 7 days or you delete time travel

Maintenance is a pipeline step, not a quarterly heroic cleanup. Lab 7 shows the details.

HANDS-ON

Lab 5 · Gold star schema

Lab 6 · Optimize

- 40 minutes together
- Site → Labs 5 and 6
- Dimension exercises have collapsible solutions - use them
- Prove EUR 2,613,940 at the consumption layer

BEFORE LAB 7 · THE PROBLEM

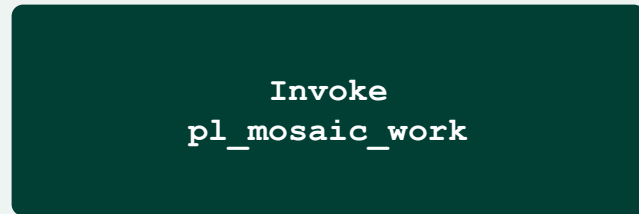
It all works because you pressed the buttons in the right order.

Ingest, then Silver, then Gold, then maintenance - from memory, by hand. Production means the chain runs itself, records that it ran, and screams when it fails.

CONCEPT

Separate the work from the watching

pl_run_all · the orchestrator (owns observability)



Log success

Log failure + error.message

*one green edge
one red edge
no ambiguity*

pl_mosaic_work · the work (knows nothing about logging)



A failure anywhere inside → the Invoke fails → the red edge fires. Tested, not assumed.

HANDS-ON

Lab 7 · One pipeline to run it all

25 minutes · Site → Lab 7 · Build both pipelines, run the chain, check the log - then break it on purpose and watch the failure

The build you would have inherited

No provenance

fixed in Lab 1

One pipeline per table

fixed in Lab 2

Full reload nightly

fixed in Lab 2

Duplicates in reporting

fixed in Lab 4

Patched aggregates

fixed in Lab 4

**Measures stored as
columns**

fixed in Lab 5

No table maintenance

fixed in Lab 6

Failure without witnesses

fixed in Lab 7

Workspace mosaic-disaster · shared read-only · the full forensics checklist is Lab 8 on the site

THE ARC OF THE DAY

At 9:00 you inherited

EUR 2,847,310

a number nobody could defend

By 17:00 you can prove

EUR 2,613,940

and show where every euro of the difference went: a renamed file · a repeating API · a late refund

The checklist you built today

- Provenance columns (`_load_timestamp`, `_source_file`, `_batch_id`) on everything that lands
- Business-key deduplication where truth is manufactured, file idempotency where it arrives
- Ingestion driven by a control table - onboarding a source is an INSERT
- Incremental aggregates driven by the union of changed keys across ALL contributing tables
- A standing reconciliation assertion that runs with every load and fails loudly
- OPTIMIZE, V-Order, VACUUM as pipeline steps, never as quarterly heroics
- One orchestrator: work separated from watching, failure path tested on purpose

Score your own platform out of 7 tomorrow.

EUR 2,613,940

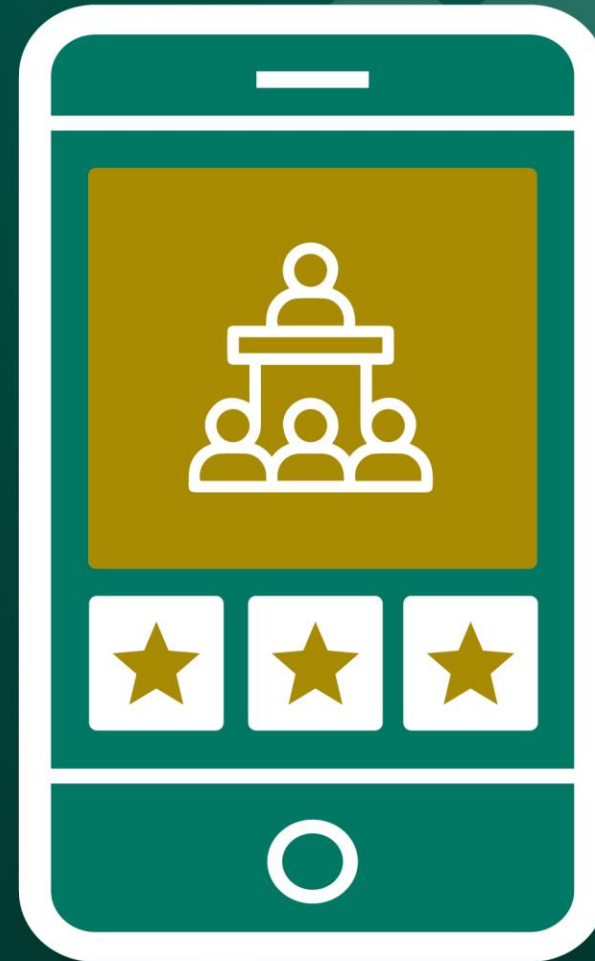
...and you can prove it.

Thank you · Nikola Ilic · Data Mozart

#FabConEurope



Please rate
this session
on the app



cvent

